

movie_sentiment_countvect

April 4, 2026

```
[1]: import nltk

# Tokenizer models (required for word_tokenize and sent_tokenize)
nltk.download('punkt')

# Newer versions of NLTK sometimes require this table for punkt tokenizer
nltk.download('punkt_tab')

# Stopword list (used to remove common words like "the", "is", "in")
nltk.download('stopwords')

# WordNet lexical database (used for lemmatization)
nltk.download('wordnet')

# WordNet multilingual dictionary mapping
nltk.download('omw-1.4')

# Part-of-Speech tagging model
# Used for identifying nouns, verbs, adjectives etc.
nltk.download('averaged_perceptron_tagger')

# Named Entity Recognition chunker model
# Used for detecting entities like PERSON, LOCATION, ORGANIZATION
nltk.download('maxent_ne_chunker')

# Word list used by the NER chunker
nltk.download('words')

# VADER sentiment lexicon
# Used for sentiment analysis (positive, negative, neutral)
nltk.download('vader_lexicon')
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data] C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

```

[nltk_data] Downloading package stopwords to
[nltk_data]   C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]   C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data]   C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]   date!
[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data]   C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data]   Package maxent_ne_chunker is already up-to-date!
[nltk_data] Downloading package words to
[nltk_data]   C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data]   Package words is already up-to-date!
[nltk_data] Downloading package vader_lexicon to
[nltk_data]   C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data]   Package vader_lexicon is already up-to-date!

```

```
[1]: True
```

```
[2]: from nltk.corpus import movie_reviews

print(movie_reviews.categories())
```

```
['neg', 'pos']
```

```
[ ]: print(movie_reviews.fileids()[:10])
```

```
[ ]: print(movie_reviews.raw('neg/cv000_29416.txt')[:])
```

```
[ ]: print(movie_reviews.raw('neg/cv001_19502.txt'))
```

```
[3]: documents = []
labels = []

for category in movie_reviews.categories():

    for fileid in movie_reviews.fileids(category):

        text = movie_reviews.raw(fileid)

        documents.append(text)
        labels.append(category)
```

```
print(len(documents))
```

2000

```
[4]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    documents,
    labels,
    test_size=0.2,
    random_state=42
)
```

```
[5]: dict1 = {'pos':0, 'neg':0}
for label in labels:
    if label == 'pos':
        dict1['pos']+=1
    else:
        dict1['neg']+=1
```

```
[ ]:
```

```
[6]: dict1
```

```
[6]: {'pos': 1000, 'neg': 1000}
```

```
[7]: import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

stop_words = set(stopwords.words('english'))
# IMPORTANT: keep negation words
negation_words = {'not', 'no', 'never', 'nor'}
stop_words = stop_words - negation_words
def clean_text(text):

    text = text.lower()

    text = re.sub(r'[\w\s]', '', text)

    tokens = word_tokenize(text)

    tokens = [w for w in tokens if w not in stop_words]

    return " ".join(tokens)
```

```
[8]: X_train = [clean_text(t) for t in X_train]
X_test = [clean_text(t) for t in X_test]
```

```
[9]: from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer(
    max_features=5000,
    min_df=2,
    max_df=0.8,
    ngram_range=(1,2)    # IMPORTANT
)

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

```
[15]: from sklearn.linear_model import LogisticRegression

model = LogisticRegression(class_weight='balanced', max_iter=200)

model.fit(X_train_vec, y_train)
```

```
[15]: LogisticRegression(class_weight='balanced', max_iter=200)
```

```
[16]: from sklearn.metrics import accuracy_score

pred = model.predict(X_test_vec)

print("Accuracy:", accuracy_score(y_test, pred))
```

Accuracy: 0.8425

```
[17]: test_review = "This movie was fantastic and the acting was great"

clean = clean_text(test_review)

vec = vectorizer.transform([clean])

prediction = model.predict(vec)

print(prediction)
```

['neg']

```
[18]: from sklearn.metrics import confusion_matrix
print(confusion_matrix(y_test, pred))
```

```
[[169  30]
 [ 33 168]]
```

```
[19]: test_review = "This movie was terrible and the acting was bad"
```

```
clean = clean_text(test_review)
```

```
vec = vectorizer.transform([clean])
```

```
prediction = model.predict(vec)
```

```
print(prediction)
```

```
['neg']
```

```
[20]: def predict_sentiment(review):
```

```
    clean = clean_text(review)
```

```
    vec = vectorizer.transform([clean])
```

```
    prediction = model.predict(vec)
```

```
    return prediction[0]
```

```
[21]: prediction = predict_sentiment("I really enjoyed this movie, but the story was  
    ↪bad and it is too lengthy!")
```

```
[22]: prediction
```

```
[22]: np.str_('neg')
```

```
[23]: prediction = predict_sentiment("I really enjoyed this movie, it was fantastic!")
```

```
[24]: prediction
```

```
[24]: np.str_('neg')
```

```
[25]: predict_sentiment("This movie was terrible and the acting was bad")
```

```
[25]: np.str_('neg')
```

```
[26]: predict_sentiment(" cinematography was beautiful,actors done great job,you will  
    ↪enjoy this movie,go for it!")
```

```
[26]: np.str_('neg')
```

```
[ ]:
```

```
[ ]:
```